

NAME : SANJAY.

ROLL NO : 23AM061

ML-II Assignment-II

III YR - CSE (AIML)



# 1 Markov Decision Process

## States ( $s$ )

Each intersection in the grid is a state.

A state can be repr as

$$S = (x, y, p)$$

where

$x, y$ : taxi's grid position

$p$ : passenger status  $\rightarrow$

0: waiting

1: onboard

2: delivered.

## Actions ( $a$ )

Possible actions at any intersection

N - Move North

S - Move South

E - Move East

W - Move West

Pick - Pick up Passenger

Drop - Drop off passenger

## Transition Probabilities ( $p$ )

Probability of reaching from  $s$  after taking action  $a$ :

$$P(s' | s, a) = \begin{cases} 1 - \epsilon & , \text{ if ent move is successful} \\ [\epsilon p] \epsilon & , \text{ if taxi slip / stays in place} \\ [\epsilon p] 0 & , \text{ otherwise.} \end{cases}$$



## Reward Function (R)

Each step -1

Successful pickup +10

Successful drop +100

Collision -100

Illegal pickup -10

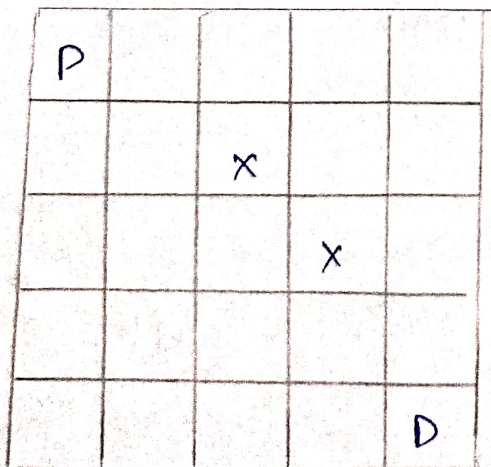
$$R(s, a, s') = \begin{cases} +100 & \text{if packet dropped successfully} \\ -100 & \text{if collision occurs} \\ -1 & \text{otherwise} \end{cases}$$

## Objective Function

Maximize expected discounted reward

$$\pi^* = \arg \max_{\pi} E_{\pi} \left[ \sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \right]$$

## Diagram



Below is a 5x5 grid diagram.



Legend :

P  $\rightarrow$  Passenger pickup point

D  $\rightarrow$  Destination

x  $\rightarrow$  obstacles.

Each intersection = State

Arrows represent actions.

## Policy Learning.

To minimize travel time & avoid collisions,

We use Q learning.

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R(s,a,s') + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where.

Learning rate (0,1)

Discount factor (0,1)

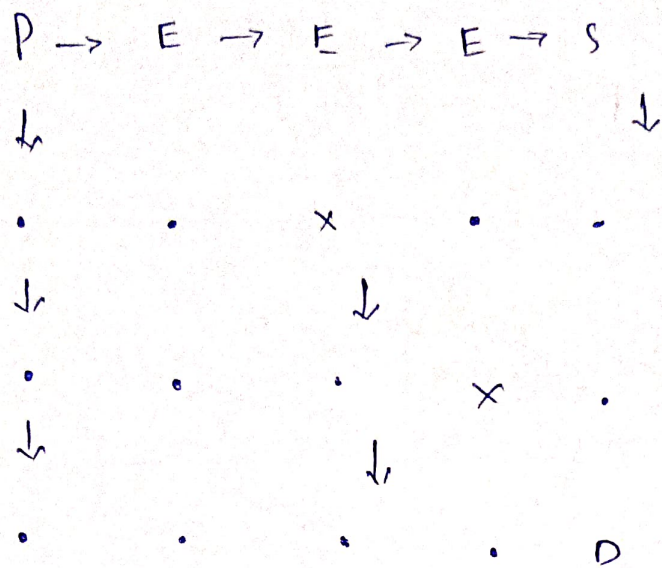
Optimal policy is then

$$\pi^*(s) = \arg \max_a Q(s,a)$$

## Learning output Example

After training, learned policy might look like :





When passenger Onboard :

Follow path toward drop off

Avoid obstacles

Drop passenger at D

Real world connection

Real autonomous Taxi Equivalent

GPS position, passenger status

Movement decision,

Trip efficiency.



## 2 Hidden Markov Model

### HMM Formulation

HMM is defined by  $(S, O, A, B, \pi)$

Hidden state,  $(s)$

discrete health states

$S = \{ \text{calm, stressed, Anxious} \}$

Observation  $O$

Continuous sensor vector at

time  $t$ :

$$O_t = [HR_t, Temp_t, Mohon_t]^T$$

Transition Matrix  $A$ :

$$a_{ij} = P(s_{t+1} = j | s_t = i)$$

Eg:

$$A = \begin{bmatrix} 0.90 & 0.80 & 0.02 \\ [4pt] 0.10 & 0.80 & 0.10 \\ [4pt] 0.05 & 0.20 & 0.75 \end{bmatrix}$$

Emission probabilities,

For continuous observation, model

$p(O_t | s_t = j)$  as a multivariate Gaussian:

$$= N(O_t; \mu_j, \Sigma_j)$$



## Initial Distribution

$$\pi = [0.8, 0.15, 0.05]$$

Starts mostly calm.

## Inference - Forward & Backward

### Initialization

$$\delta_1(j) = \log \pi_j + \log p(o_1 | s_1 = j)$$

### Recursion

$$\delta_t(j) = \max_i [\delta_{t-1}(i) + \log a_{ij}] + \log p(o_t | s_t = j)$$

## Posterior Marginals

Compute  $\alpha$  - Forward

$\beta$  - Backward

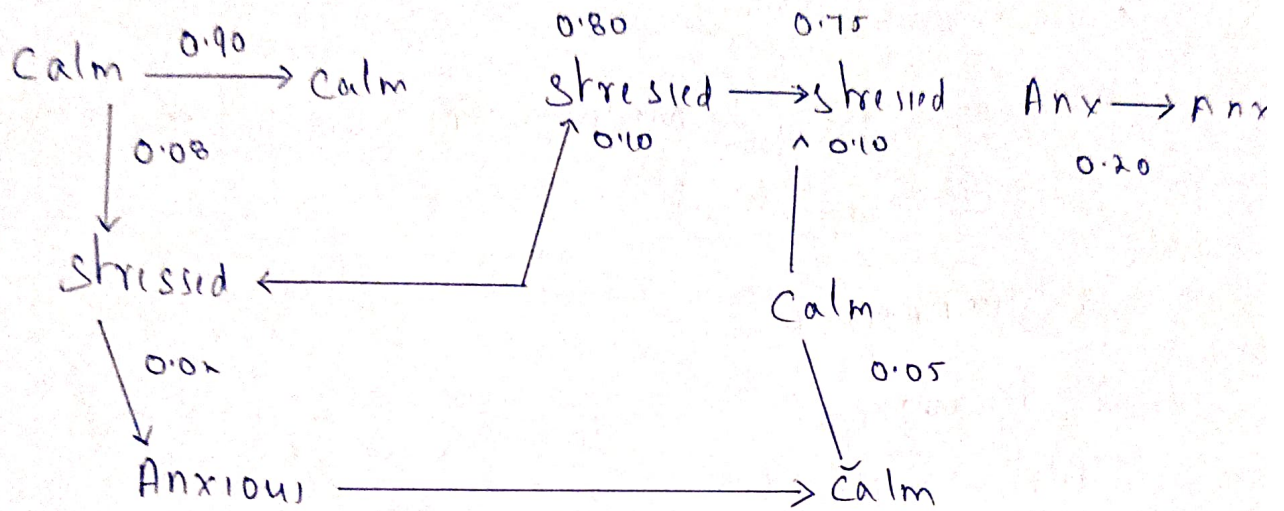
$$\alpha_1(j) = \pi_j p(o_1 | s_1 = j)$$

$$\beta_T(j) = 1$$

$$\gamma_t(j) = p(s_t = j | o_{1:T}) \propto \alpha_t(j) \beta_t(j)$$



### 3-state HMM diagram



### CODE SNIPPER

Health Monitoring

```
import numpy as np
```

```
# Hidden States
```

```
States = ["Calm", "Stressed", "Anx"]
```

```
n = len(States)
```

```
# Transition probabilities
```

```
A = np.array([  
    [0.9, 0.08, 0.02],  
    [0.1, 0.6, 0.3],  
    [0.1, 0.3, 0.6]  
])
```

```
)
```



# initial state prob

```
pi = np.array([0.7, 0.2, 0.1])
```

# observation sequence

# 0 = Low HR, 1 = Medium HR, 2 = High R.

```
O = [0, 1, 1, 2, 1, 0]
```

# VITERBI ALG

```
T = len(O)
```

```
delta = np.zeros((T, n))
```

```
psi = np.zeros((T, n), int)
```

# initialization

```
delta[0] = np.log(pi) + np.log([B[:, O[0]]])
```

# Result.

```
decoded = [states[i] for i in states_seq]
```

```
print("Most likely hidden states:")
```

```
print(decoded)
```

OUTPUT

Most likely hidden states:

```
['calm', 'stressed', 'stressed', 'Anx']
```



3

## MODEL SELECTION IN HMM

### Description

Daily market trends using an HMM, where hidden states represent latent market regimes and observations are market indicators, such as

- \* Daily return percentage
- \* Trading volume
- \* Volatile index
- \* Moving average signals.

### Why model selection matters

- Too few states  $\rightarrow$

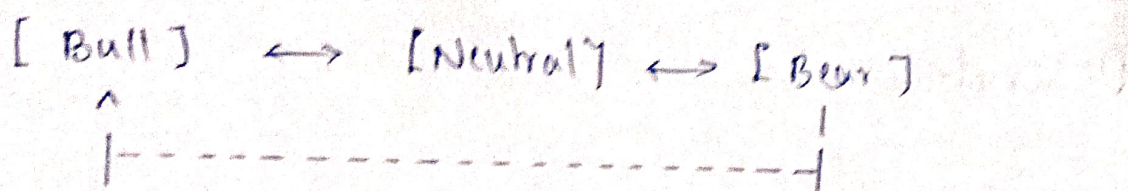
underfitting: model cannot capture all distinct market behaviours.

- Too many states  $\rightarrow$

overfitting: model memorises noise instead of trends, reducing predictive generalization.



## Conceptual HMM



Each state emits observation,

Bull  $\rightarrow$  high returns

low volatility

Neutral  $\rightarrow$  Small returns

moderate volume

Bear  $\rightarrow$  negative returns

high volume.

## Eval Candidate Models

Multiple HMMs with different states counts - (eg:  $k = 2, 3, 4, 5$ )

Eval using AIC, BIC, Cross-validation

## Criteria for Model Selection

$$AIC = 2k - 2\ln(L)$$

$$BIC = k\ln(n) - 2\ln(L)$$

$$CV = \frac{1}{F} \sum_{f=1}^F (n \cdot D(O_{val}^{(f)} | \text{model } k))$$



## Step by Step Procedure

Train HMMs with  $K = 2, 3, 4, 5$

Compute log likelihood on data

Compute AIC & BIC for each  $K$

Run 5-fold CV & compute average validation score.

Compare Results & pick model.

## Intuitive Explanation

Increasing  $K$  (more states)

Improving training accuracy

But  $\rightarrow$  model may invent states

## Benefits

Detecting shifts helps in portfolio rebalancing and risk control.

HMM based regime detection combined with model selection ensures robust performance.

Prevents false alarms



## 4 Q-Learning.

Application Warehouse robot.

States: compact, enam friendly state

representation include:

Robot grid position:  $(n, y)$ ,  $n, y \in \{0, 1, 2\}$

Carrying status:  $c \in \{0, 1\}$

$S = (n, y, c)$

no of states =  $3 \times 3 \times 2 = 18$

Actions (A)

Low-Level discrete moves + task actions:

$A = \{ \text{North, East, South, West, Pick, Drop} \}$

Transition Dynamics

Pick succeeds only if robot at pickup cell and  $c=0$ ; then  $c \rightarrow 1$

Drop succeeds only if robot at drop cell &  $c \rightarrow 0$  & may end or continue for next task.



## Reward Function

Step penalty (time / energy)

$$\gamma_{\text{step}} = -1 \text{ per time step}$$

Movement energy cost:

optionally add small additional negative

reward per move proportional to distance.

Successful pickup: +0 or small +1

Successful dropout: +100

Collision action: -10

Battery penalty or long run penalty for energy can be modeled if required.

## Example

$$R(s, a, s') = \begin{cases} +100 & \text{successful drop} \\ -10 & \text{illegal drop} \\ -1 & \text{otherwise} \end{cases}$$

## Update rule

$$Q(s, a) \leftarrow Q(s, a) + \alpha (r + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

where,

$Q(s, a)$  = action-value estimate



$\alpha$  = learning rate  $\in [0, 1]$ ,

$\gamma$  = discount factor  $\in [0, 1]$ ,

## Convergence & Parameter Choice

$0 < \alpha \leq 1$ : Small but not too small

helps stability. For guaranteed convergence in some theory.

$\gamma$ : Choose near 1 if long term reward

is important, lower if immediate energy matters more.

## Real world consideration.

\* Add safety layers to block collisions.

Use constrained RL for energy tradeoffs.

\* Combine time and energy or use pareto optimization.

\* Train in simulator with domain randomization